

DAY 4

PostgreSQL 18 Backup & Disaster Recovery

LIVE Patroni / Zalando Postgres Operator Cluster

UNIFIED SINGLE-LINE COMMAND EDITION

Namespace	postgres-ha
Cluster	acid-postgres-ha (Zalando Operator + Patroni)
PostgreSQL	18
Kubernetes	Docker Kubernetes (not GKE)
GCS bucket	postgres-ha-backups-parameswari-2026
Backup tools	pg_dump, pg_dumpall, pg_basebackup, WAL archiving, pgBackRest
Enterprise features	Full / Incremental / Differential / Compression / Encryption / Retention / PITR
Restore rule	Isolated recovery storage or a separate recovery cluster only
Sources merged	READABLE_COPYPASTE_FINAL + CORRECTED_6/8/11 + Kubernetes backup-lab

CRITICAL SAFETY RULE

Take backups from the live cluster, but never overwrite the active Patroni member PVCs during restore or PITR. Perform physical restore only on isolated recovery storage or a separate recovery cluster.

COPY/PASTE RULE

Every shell command below is one physical command line. Configuration files are shown separately and are not shell commands. Run commands one at a time. Do not paste two commands onto the same shell line unless they are intentionally joined with &&.

Table of Contents

0. Lab Architecture, Scope and Safety
1. Pre-Lab Validation of the Live Patroni Cluster
2. Backup Concepts: Logical, Physical, WAL and PITR
3. Lab 1 — Logical Backup from the Live Cluster
4. Lab 2 — Physical Backup with pg_basebackup (Corrected)
5. Lab 3 — WAL Archiving Validation
6. Lab 4A — Custom Spilo Image (Docker build & Docker Hub push)
7. Lab 4B — GCS Bucket, IAM and pgBackRest Configuration
8. Lab 5 — Full Backup with pgBackRest
9. Lab 6 — Incremental Backup
10. Lab 7 — Differential Backup
11. Repository, Compression, Encryption and Retention
12. Lab 8 — Isolated Restore / Full Cluster Recovery

13. Lab 9 — Point-in-Time Recovery (PITR)
14. Patroni Failover Considerations During Backup
15. Optional Disposable backup-lab (safe pgBackRest playground)
16. Validation, Monitoring, Troubleshooting and Cleanup
17. RPO / RTO and Suggested Backup Schedule
18. Final Coverage Checklist

Implementation note

Zalando/Spilo commonly uses WAL-G/WAL-E for native backup. pgBackRest is not automatically present in all Spilo images. Do not install packages ad hoc into a running production pod and assume they will survive restart. Build an immutable image that contains a PostgreSQL 18-compatible pgBackRest, push it to a registry, then point the cluster at that image.

0. Lab Architecture, Scope and Safety

0.1 What runs live and what is isolated?

- **LIVE:** acid-postgres-ha remains Patroni-managed and continues serving traffic during backup exercises.
- **LIVE BACKUP:** logical dumps, pg_basebackup, WAL archive checks and pgBackRest backup commands run against the current primary or an eligible replica.
- **ISOLATED RESTORE:** physical restore and PITR use separate recovery storage. Never replace files below active PGDATA while Patroni is running.
- **REPOSITORY:** durable object storage (GCS) — bucket postgres-ha-backups-parameswari-2026.

0.2 Recovery architecture

```
Application -> acid-postgres-ha Service -> Patroni Primary
Primary -> logical / physical / WAL / pgBackRest repository
Repository + archived WAL -> isolated recovery PGDATA -> validation -> controlled cutover
```

0.3 Why failover is not backup

Patroni protects availability. It does not protect against logical mistakes such as DROP TABLE, accidental DELETE, bad application updates, corruption that propagates to replicas, or retention mistakes. Backup and PITR provide recovery from those events.

1. Pre-Lab Validation of the Live Patroni Cluster

1.1 Set namespace and inspect resources

```
kubectl config set-context --current --namespace=postgres-ha
kubectl get pods -o wide
kubectl get svc
kubectl get postgresql
kubectl get pvc
kubectl get statefulset
```

1.2 Set variables

```
NS=postgres-ha
CLUSTER=acid-postgres-ha
```

1.3 Identify the current primary — do not assume pod-0

```
for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do echo "=== $p ===";
  kubectl -n "$NS" exec "$p" -- curl -s http://127.0.0.1:8008/ | jq -r '"role=" + (.role //
  "unknown") + " state=" + (.state // "unknown")'; done
```

Exactly one member should report primary/leader. If the cluster is unhealthy, stop the backup lab and repair HA first.

1.4 Set PRIMARY and REPLICA after checking

```
PRIMARY=acid-postgres-ha-0
REPLICA=acid-postgres-ha-1
```

Change both values to match the actual current primary and a healthy replica.

1.5 Validate role, version, PGDATA and LSN

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SELECT pg_is_in_recovery(),
    current_database(), version();"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SHOW data_directory;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT now(), pg_current_wal_lsn(),
    pg_is_in_recovery();"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT application_name, client_addr,
    state, sync_state, sent_lsn, write_lsn, flush_lsn, replay_lsn FROM pg_stat_replication;"
```

2. Backup Concepts: Logical, Physical, WAL and PITR

Logical backup (`pg_dump` / `pg_dumpall`) stores SQL objects and data and supports database/object-level restore. Physical backup (`pg_basebackup` / `pgBackRest`) copies the cluster data files and is the foundation for cluster-level recovery and PITR. WAL archiving continuously preserves transaction history. PITR restores a physical backup and replays archived WAL up to a selected time / LSN / transaction.

2.1 Full vs differential vs incremental

- **Full:** complete physical backup; establishes an independent backup baseline.
- **Differential:** changes since the most recent full backup. Restore uses that full + the chosen differential + required WAL.
- **Incremental:** changes since the immediately preceding backup. Restore can require a chain of dependent backups + required WAL.
- Archived WAL is not a substitute for a base backup; it is the continuous roll-forward stream used after restoring a physical backup.

2.2 RPO and RTO

RPO is acceptable data loss. Frequent, healthy WAL archiving reduces potential data loss. RTO is acceptable service restoration time; it depends on backup size, storage/network speed, number of backup dependencies, WAL replay volume and validation time.

3. Lab 1 — Logical Backup from the Live Cluster

3.1 Create local backup directory

```
mkdir -p backups/logical
```

3.2 Full cluster logical backup

```
kubectl -n "$NS" exec "$PRIMARY" -- pg_dumpall -U postgres --clean --if-exists >
    "backups/logical/${CLUSTER}-dumpall-$(date +%Y%m%d-%H%M%S).sql"
ls -lh backups/logical/
```

3.3 Custom-format backup of banking database

```
kubectl -n "$NS" exec "$PRIMARY" -- pg_dump -U postgres -Fc -d banking -f /tmp/banking.dump
kubectl -n "$NS" cp "$PRIMARY:/tmp/banking.dump" "backups/logical/banking-$(date
    +%Y%m%d-%H%M%S).dump"
```

3.4 Verify without restoring

```
BACKUP_FILE=$(ls -lt backups/logical/banking-*.dump | head -1)
echo "$BACKUP_FILE"
pg_restore -l "$BACKUP_FILE" | head -40
SQL_FILE=$(ls -lt backups/logical/${CLUSTER}-dumpall-*.sql | head -1)
head -30 "$SQL_FILE"
```

3.5 Test logical restore into an isolated test database

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "DROP DATABASE IF EXISTS
banking_restore_test;"

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "CREATE DATABASE banking_restore_test;"
BACKUP_FILE=$(ls -lt backups/logical/banking-*.dump | head -1)
echo "$BACKUP_FILE"

kubectl -n "$NS" cp "$BACKUP_FILE" "$PRIMARY:/tmp/banking-restore.dump"

kubectl -n "$NS" exec "$PRIMARY" -- pg_restore -U postgres -d banking_restore_test --clean
--if-exists /tmp/banking-restore.dump

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking_restore_test -c "\dt"
```

On a real production system, run restore tests in a dedicated recovery cluster instead of consuming I/O/storage on the production primary.

4. Lab 2 — Physical Backup with pg_basebackup (Corrected)

pg_basebackup uses the PostgreSQL replication protocol. A normal superuser login is not enough if pg_hba.conf does not allow that user for replication. In this Zalando/Patroni cluster, use the cluster primary Service and the operator-managed replication/standby credentials.

4.1 Verify the primary Service name

```
kubectl -n "$NS" get svc
PRIMARY_SERVICE=acid-postgres-ha
kubectl -n "$NS" exec "$REPLICA" -- getent hosts "$PRIMARY_SERVICE"
```

Do not use -h 127.0.0.1 when the client command is running in a replica pod. 127.0.0.1 means that same replica pod. For this cluster the primary Service is acid-postgres-ha.

4.2 Inspect replication state and lag

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT application_name, client_addr,
state, sync_state, sent_lsn, write_lsn, flush_lsn, replay_lsn,
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_lag FROM
pg_stat_replication;"
```

4.3 Identify replication-capable roles

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT rolname, rolreplication,
rolcanlogin FROM pg_roles WHERE rolreplication = true;"
```

4.4 Check replication HBA rules

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT line_number, type, database,
user_name, address, auth_method, error FROM pg_hba_file_rules WHERE 'replication' =
ANY(database);"

```

Do not add broad rules such as host replication all 0.0.0.0/0 trust.

4.5 Find the operator-managed replication/standby secret

```
kubectl -n "$NS" get secrets | grep -Ei "standby|repl"
REPL_SECRET=standby.acid-postgres-ha.credentials.postgresql.acid.zalan.do
```

4.6 Load replication username and password into your local shell

```
REPL_USER=$(kubectl -n "$NS" get secret "$REPL_SECRET" -o jsonpath='{.data.username}' | base64
-d)
REPL_PASS=$(kubectl -n "$NS" get secret "$REPL_SECRET" -o jsonpath='{.data.password}' | base64
-d)
echo "$REPL_USER"
```

Do not echo the password. Keep it only in the shell variable and unset it after the exercise.

4.7 Test ordinary connectivity with the replication user

```
kubectl -n "$NS" exec "$REPLICA" -- env PGPASSWORD="$REPL_PASS" psql -h "$PRIMARY_SERVICE" -p
5432 -U "$REPL_USER" -d postgres -c "SELECT pg_is_in_recovery(), inet_server_addr(),
inet_server_port();"

```

Expected: `pg_is_in_recovery()` = false because acid-postgres-ha routes to the current primary.

4.8 Prepare backup directory in the client pod

```
kubectl -n "$NS" exec "$REPLICA" -- rm -rf /tmp/basebackup
kubectl -n "$NS" exec "$REPLICA" -- mkdir -p /tmp/basebackup

```

4.9 Run pg_basebackup with the correct host and replication user

```
kubectl -n "$NS" exec "$REPLICA" -- env PGPASSWORD="$REPL_PASS" pg_basebackup -h
"$PRIMARY_SERVICE" -p 5432 -U "$REPL_USER" -D /tmp/basebackup -Ft -z -Xs -P

```

Options: -Ft = tar format; -z = gzip; -Xs = stream WAL while the backup runs; -P = progress display.

4.10 Verify physical backup files

```
kubectl -n "$NS" exec "$REPLICA" -- ls -lh /tmp/basebackup
kubectl -n "$NS" exec "$REPLICA" -- find /tmp/basebackup -maxdepth 1 -type f -printf "%f %s
bytes\n"

```

4.11 Copy the backup out to the workstation

```
mkdir -p backups/physical
PHYSICAL_DIR="backups/physical/basebackup-$(date +%Y%m%d-%H%M%S)"
kubectl -n "$NS" cp "$REPLICA:/tmp/basebackup" "$PHYSICAL_DIR"
find "$PHYSICAL_DIR" -maxdepth 2 -type f -ls | head -20

```

4.12 Remove password variable from the shell

```
unset REPL_PASS

```

4.13 Troubleshooting

- could not translate host name "postgres-ha": use PRIMARY_SERVICE=acid-postgres-ha.
- no pg_hba.conf entry for replication connection user "postgres": use the operator-managed replication/standby role.
- SSL / no encryption HBA mismatch: fix role/HBA/authentication; do not bypass with trust.

5. Lab 3 — WAL Archiving Validation

5.1 Inspect archive settings

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW archive_mode;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW archive_command;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW archive_library;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -x -c "SELECT * FROM pg_stat_archiver;"

```

5.2 Force a WAL switch and verify progress

```
BEFORE=$(kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SELECT archived_count FROM
pg_stat_archiver;")
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT pg_switch_wal();"
sleep 10
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT archived_count, failed_count,
last_archived_wal, last_archived_time, last_failed_wal, last_failed_time FROM
pg_stat_archiver;"
echo "Archived before test: $BEFORE"

```

5.3 Check native Spilo backup tooling before changing archive_command

```
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'command -v wal-g || true; command -v wal-e || true;
env | grep -E "(USE_WALG|WALE|AWS_|GS_|WAL_)" || true'
kubectl get postgresql acid-postgres-ha -o yaml | grep -A 40 -E "backup|wal|archive|clone"

```

5.4 Optional: trigger operator native backup if WAL-G is already configured

```
kubectl annotate postgresql acid-postgres-ha force-backup.acid.zalan.do=$(date +%s) --overwrite
kubectl logs -l application=spilo -c postgres --tail=100 | grep -iE 'backup|wal-g|archive'
```

6. Lab 4A — Custom Spilo Image (Docker build & Docker Hub push)

pgBackRest is not present in every Spilo image. Build an immutable image from the live cluster image, install a PostgreSQL 18-compatible pgBackRest, push it to Docker Hub (or another registry), then roll the cluster onto that image. Do not apt-get install inside a running production pod.

6.1 Record the current live image

```
CURRENT_IMAGE=$(kubectl -n "$NS" get pod "$PRIMARY" -o
  jsonpath='{.spec.containers[?(@.name=="postgres")].image}')
echo "$CURRENT_IMAGE"
```

6.2 Set image variables (change the Docker Hub namespace)

```
DOCKERHUB_USER=YOUR_DOCKERHUB_USER
CUSTOM_IMAGE=${DOCKERHUB_USER}/spilo-pgbackrest:pg18
```

Replace `YOUR_DOCKERHUB_USER` with your actual Docker Hub account. For a private registry use `registry.example.com/spilo-pgbackrest:pg18` instead.

6.3 Write the Dockerfile

File: Dockerfile.spilo-pgbackrest

```
cat > Dockerfile.spilo-pgbackrest <<'EOF'
ARG BASE_IMAGE
FROM ${BASE_IMAGE}
USER root
RUN set -eux; \
  if command -v apt-get >/dev/null 2>&1; then \
    apt-get update; \
    apt-get install -y --no-install-recommends pgbackrest ca-certificates; \
    rm -rf /var/lib/apt/lists/*; \
  else \
    echo "Adapt this Dockerfile to the base image OS"; exit 1; \
  fi; \
  pgbackrest version
USER postgres
EOF
```

6.4 Build the image from the live base

```
docker build --build-arg BASE_IMAGE="$CURRENT_IMAGE" -f Dockerfile.spilo-pgbackrest -t
  "$CUSTOM_IMAGE" .
docker images | grep spilo-pgbackrest
```

6.5 Log in to Docker Hub and push

```
docker login
docker push "$CUSTOM_IMAGE"
```

If the cluster nodes cannot pull from Docker Hub anonymously, create a pull Secret and reference it from the postgresql CR (spec.imagePullSecrets).

```
kubectl -n "$NS" create secret docker-registry dockerhub-pull
  --docker-server=https://index.docker.io/v1/ --docker-username="$DOCKERHUB_USER"
  --docker-password="$DOCKERHUB_TOKEN" --docker-email="$DOCKERHUB_EMAIL" --dry-run=client -o
  yaml | kubectl apply -f -
```

6.6 Point the live cluster at the custom image

File: pgbackrest-image-patch.json

```
cat > pgbackrest-image-patch.json <<EOF
{
  "spec": {
    "dockerImage": "${CUSTOM_IMAGE}"
  }
}
```

```
}
}
EOF
```

Because the JSON file is static, either substitute the image name by hand or generate the patch after CUSTOM_IMAGE is set:

```
python3 - <<PY
import json, os
print(json.dumps({"spec":{"dockerImage": os.environ["CUSTOM_IMAGE"]}}, indent=2))
PY > pgbackrest-image-patch.json

kubectl -n "$NS" get postgresql "$CLUSTER" -o yaml > "${CLUSTER}-before-image.yaml"

kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file
pgbackrest-image-patch.json --dry-run=server -o yaml > /tmp/image-preview.yaml

kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file
pgbackrest-image-patch.json

kubectl -n "$NS" rollout status statefulset/"$CLUSTER" --timeout=600s
```

6.7 Verify pgBackRest on every member

```
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do printf "%s: " "$pod";
kubectl -n "$NS" exec "$pod" -c postgres -- pgbackrest version; done
```

Expected: all three pods report a PostgreSQL 18-compatible pgBackRest (lab target 2.59.1). If apt reports that pgBackRest does not support PostgreSQL 18, stop and use a different build.

If the current image does not use apt, adapt the Dockerfile to its OS (apk, yum, or a custom build of pgBackRest from source). Every Patroni member must use the same immutable image.

7. Lab 4B — GCS Bucket, IAM and pgBackRest Configuration

Use one pgBackRest identity everywhere: config /etc/pgbackrest/pgbackrest.conf and stanza postgres-ha. Mixing this with /home/postgres/pgdata/pgbackrest.conf or stanza acid-postgres-ha causes WAL archive timeouts.

Namespace	postgres-ha
Cluster / pods	acid-postgres-ha-0 / -1 / -2
Kubernetes	Docker Kubernetes (not GKE)
pgBackRest	2.59.1 (from custom image)
GCS bucket	postgres-ha-backups-parameswari-2026
Stanza	postgres-ha
Config path	/etc/pgbackrest/pgbackrest.conf
Cipher Secret	pgbackrest-repo-secret → /etc/pgbackrest-secret/repo-cipher-pass
GCS key Secret	pgbackrest-gcs-key → /etc/pgbackrest-gcs/gcs-key.json

7.1 Set variables and save the current CR

```
NS=postgres-ha
CLUSTER=acid-postgres-ha
STANZA=postgres-ha
PGBR_CONF=/etc/pgbackrest/pgbackrest.conf
BUCKET=postgres-ha-backups-parameswari-2026
kubectl -n "$NS" get pods -o wide
kubectl -n "$NS" get postgresql "$CLUSTER" -o yaml > "${CLUSTER}-before-pgbackrest.yaml"
```

7.2 Create the GCS bucket (if it does not already exist)

```
PROJECT_ID=$(gcloud config get-value project)
echo "$PROJECT_ID"
REGION=asia-south1
gcloud storage buckets describe "gs://$BUCKET" >/dev/null 2>&1 || gcloud storage buckets create
"gs://$BUCKET" --project="$PROJECT_ID" --location="$REGION" --uniform-bucket-level-access
```



```
gcloud storage buckets describe "gs://$BUCKET" --format="value(name,location,storageClass)"
gcloud storage ls "gs://$BUCKET"
```

7.3 Create a dedicated service account and grant objectAdmin on the bucket

Docker Kubernetes cannot use `repo1-gcs-key-type=auto` because `metadata.google.internal` is unavailable. Use a mounted service-account JSON key.

```
SA_NAME=pgbackrest-gcs
SA_EMAIL="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"
echo "$SA_EMAIL"

gcloud iam service-accounts describe "$SA_EMAIL" >/dev/null 2>&1 || gcloud iam service-accounts
  create "$SA_NAME" --display-name="pgBackRest GCS backup"

gcloud storage buckets add-iam-policy-binding "gs://$BUCKET"
  --member="serviceAccount:${SA_EMAIL}" --role="roles/storage.objectAdmin"
```

7.4 If key creation is blocked by Organization Policy

Google Cloud Console → IAM & Admin → Organization Policies. Open the Active `iam.disableServiceAccountKeyCreation` constraint. At project level, override the parent and set enforcement to Off / Not enforced. Wait for propagation, then continue.

```
gcloud org-policies describe constraints/iam.disableServiceAccountKeyCreation
  --project="$PROJECT_ID"

test -f pgbackrest-gcs-key.json || gcloud iam service-accounts keys create
  pgbackrest-gcs-key.json --iam-account="$SA_EMAIL"

chmod 600 pgbackrest-gcs-key.json

kubectl -n "$NS" get secret pgbackrest-gcs-key >/dev/null 2>&1 || kubectl -n "$NS" create
  secret generic pgbackrest-gcs-key --from-file=gcs-key.json=./pgbackrest-gcs-key.json

kubectl -n "$NS" get secret pgbackrest-gcs-key -o jsonpath="{.data.gcs-key\.json}" >/dev/null
  && echo "gcs-key.json OK"
```

7.5 Preserve the existing cipher Secret

Do not recreate `pgbackrest-repo-secret` if it already exists. Replacing the passphrase can make encrypted backups unreadable.

```
kubectl -n "$NS" get secret pgbackrest-repo-secret

kubectl -n "$NS" get secret pgbackrest-repo-secret >/dev/null 2>&1 || kubectl -n "$NS" create
  secret generic pgbackrest-repo-secret --from-literal=repo-cipher-pass="$(openssl rand
  -base64 32)"

kubectl -n "$NS" get secret pgbackrest-repo-secret -o jsonpath="{.data.repo-cipher-pass}"
  >/dev/null && echo "repo-cipher-pass OK"
```

7.6 Discover the current primary and PGDATA

```
PRIMARY=$(for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do v=$(kubectl
  -n "$NS" exec "$pod" -c postgres -- psql -U postgres -Atqc "select pg_is_in_recovery();" );
  if [ "$v" = "f" ]; then echo "$pod"; break; fi; done); echo "$PRIMARY"

PGDATA_ACTUAL=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -Atqc "SHOW
  data_directory;"); echo "$PGDATA_ACTUAL"
```

Expected PGDATA is normally `/home/postgres/pgdata/pgroot/data`.

7.7 Create `pgbackrest.conf`

File: `pgbackrest.conf`

```
cat > pgbackrest.conf <<'EOF'
[postgres-ha]
pg1-path=/home/postgres/pgdata/pgroot/data
pg1-port=5432

[global]
repo1-type=gcs
repo1-gcs-bucket=postgres-ha-backups-parameswari-2026
repo1-gcs-key-type=service
repo1-gcs-key=/etc/pgbackrest-gcs-key.json
repo1-path=/pgbackrest/postgres-ha
```



```

repo1-retention-full=2
repo1-retention-diff=4
repo1-cipher-type=aes-256-cbc
compress-type=zst
compress-level=3
process-max=2
start-fast=y
archive-timeout=120
log-level-console=info

[global:archive-push]
process-max=2

[global:archive-get]
process-max=2
EOF

```

If PGDATA_ACTUAL differs, change pg1-path before creating the ConfigMap.

```

kubectl -n "$NS" create configmap pgbackrest-config
  --from-file=pgbackrest.conf=./pgbackrest.conf --dry-run=client -o yaml | kubectl apply -f -
kubectl -n "$NS" get configmap pgbackrest-config -o jsonpath="{.data.pgbackrest\.conf}" | grep
  -E "repo1-gcs-bucket|repo1-gcs-key-type|repo1-gcs-key|repo1-path|archive-timeout"

```

7.8 Mount config, cipher Secret and GCS key

File: pgbackrest-additional-volumes-patch.json

```

cat > pgbackrest-additional-volumes-patch.json <<'EOF'
{
  "spec": {
    "additionalVolumes": [
      {
        "name": "pgbackrest-config", "mountPath": "/etc/pgbackrest", "targetContainers": ["postgres"],
        "volumeSource": {"configMap": {"name": "pgbackrest-config"}}},
      {
        "name": "pgbackrest-secret", "mountPath": "/etc/pgbackrest-secret", "targetContainers": ["postgres"],
        "volumeSource": {"secret": {"secretName": "pgbackrest-repo-secret"}}},
      {
        "name": "pgbackrest-gcs-key", "mountPath": "/etc/pgbackrest-gcs", "targetContainers": ["postgres"],
        "volumeSource": {"secret": {"secretName": "pgbackrest-gcs-key"}}}
    ]
  }
}
EOF

python3 -m json.tool pgbackrest-additional-volumes-patch.json >/dev/null && echo "JSON OK"

kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file
  pgbackrest-additional-volumes-patch.json --dry-run=server -o yaml >
  /tmp/pgbackrest-volumes-preview.yaml

kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file
  pgbackrest-additional-volumes-patch.json

kubectl -n "$NS" rollout status statefulset/"$CLUSTER" --timeout=300s

for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do printf "%s: " "$pod";
  kubectl -n "$NS" exec "$pod" -c postgres -- sh -lc 'test -r /etc/pgbackrest/pgbackrest.conf
  && test -r /etc/pgbackrest-secret/repo-cipher-pass && test -r
  /etc/pgbackrest-gcs/gcs-key.json && echo OK || echo FAILED'; done

```

Expected: every pod prints OK.

7.9 Create the pgBackRest stanza on the current primary

```

PRIMARY=$(for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do v=$(kubectl
  -n "$NS" exec "$pod" -c postgres -- psql -U postgres -Atqc "select pg_is_in_recovery();");
  if [ "$v" = "f" ]; then echo "$pod"; break; fi; done); echo "$PRIMARY"

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export
  PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
  --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha stanza-create'

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export
  PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
  --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'

```

If stanza-create says unable to find primary cluster, the command was run on a replica. Re-detect PRIMARY.

7.10 Correct PostgreSQL archive_command

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -Atqc "show archive_mode; show archive_command;"
```

File: pgbackrest-archive-command-patch.json

```
cat > pgbackrest-archive-command-patch.json <<'EOF'
```

```
{
  "spec": {
    "postgresql": {
      "parameters": {
        "archive_mode": "on",
        "archive_command": "PGBACKREST_REP01_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass) pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha archive-push %p"
      }
    }
  }
}
EOF
```

```
python3 -m json.tool pgbackrest-archive-command-patch.json >/dev/null && echo "JSON OK"
```

```
kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file pgbackrest-archive-command-patch.json --dry-run=server -o yaml > /tmp/archive-command-preview.yaml
```

```
kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file pgbackrest-archive-command-patch.json
```

```
kubectl -n "$NS" rollout status statefulset/"$CLUSTER" --timeout=300s
```

```
PRIMARY=$(for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do v=$(kubectl -n "$NS" exec "$pod" -c postgres -- psql -U postgres -Atqc "select pg_is_in_recovery();"); if [ "$v" = "f" ]; then echo "$pod"; break; fi; done); echo "$PRIMARY"
```

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -Atqc "show archive_command;"
```

Expected: `archive_command` contains `/etc/pgbackrest/pgbackrest.conf`, stanza `postgres-ha` and `archive-push %p`. It must not use `/bin/true`, `/home/postgres/pgdata/pgbackrest.conf`, or stanza `acid-postgres-ha`.

7.11 Run pgBackRest check and prove WAL archiving works

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REP01_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass); pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha check'

BEFORE=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -Atqc "select archived_count from pg_stat_archiver;"); echo "$BEFORE"

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -c "select pg_switch_wal();"

sleep 10

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -x -c "select archived_count,failed_count,last_archived_wal,last_archived_time,last_failed_wal,last_failed_time from pg_stat_archiver;"

gcloud storage ls "gs://$BUCKET/pgbackrest/postgres-ha/" | head
```

Expected: `archived_count` increases and there is no new archive failure. Only then continue to Lab 5.

8. Lab 5 — Full Backup with pgBackRest

Do not start this lab until section 7.11 succeeds.

```
PRIMARY=$(for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do v=$(kubectl -n "$NS" exec "$pod" -c postgres -- psql -U postgres -Atqc "select pg_is_in_recovery();"); if [ "$v" = "f" ]; then echo "$pod"; break; fi; done); echo "$PRIMARY"

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REP01_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass); pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --type=full backup'

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REP01_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass); pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'
```

Expected: the newest backup label ends with F and status is OK.

8.1 Create test data after the full backup

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -d banking -c "create table if
```

```
not exists dr_lab_events(id bigint generated always as identity primary key,event_time
timestampz default clock_timestamp(),note text); insert into dr_lab_events(note) values
('after full backup'); select * from dr_lab_events order by id;"
```

9. Lab 6 — Incremental Backup

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -d banking -c "insert into
dr_lab_events(note) values ('before incremental backup');"

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export
PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
--config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --type=incr backup'

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export
PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
--config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'
```

Expected: the newest backup label ends with I.

10. Lab 7 — Differential Backup

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -d banking -c "insert into
dr_lab_events(note) values ('before differential backup');"

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export
PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
--config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --type=diff backup'

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export
PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
--config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'
```

Expected: the newest backup label ends with D.

11. Repository, Compression, Encryption and Retention

- Repository: gs://postgres-ha-backups-parameswari-2026/pgbackrest/postgres-ha
- Compression: zst level 3
- Encryption: AES-256-CBC
- Retention: 2 full / 4 differential
- Archive timeout: 120 seconds

Keep the original cipher passphrase. Losing or replacing it makes encrypted backups unrecoverable.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export
PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
--config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export
PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
--config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --dry-run expire'

gcloud storage du -s "gs://$BUCKET/pgbackrest/postgres-ha"
```

12. Lab 8 — Isolated Restore / Full Cluster Recovery

Never restore into active acid-postgres-ha PVCs. Use isolated recovery storage or a separate recovery cluster.

12.1 Preferred: separate Zalando recovery cluster for logical proof

File: acid-recovery-test.yaml

```
cat > acid-recovery-test.yaml <<'EOF'
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: acid-recovery-test
  namespace: postgres-ha
spec:
  teamId: "acid"
  volume:
```

```

    size: 10Gi
  numberOfInstances: 1
  users:
    zalando: []
  databases:
    recoverydb: zalando
  postgresql:
    version: "18"
EOF
kubectl apply -f acid-recovery-test.yaml
kubectl wait --for=condition=Ready pod -l cluster-name=acid-recovery-test --timeout=10m
RECOVERY_POD=$(kubectl get pod -l cluster-name=acid-recovery-test -o
  jsonpath='{.items[0].metadata.name}')
echo "Recovery pod: $RECOVERY_POD"

```

Logical restore into the test cluster (safe, no PVC overwrite):

```

BACKUP_FILE=$(ls -lt backups/logical/banking-*.dump | head -1)
kubectl -n "$NS" cp "$BACKUP_FILE" "$RECOVERY_POD:/tmp/banking.dump"
kubectl -n "$NS" exec "$RECOVERY_POD" -- pg_restore -U postgres -d recoverydb -v --clean
  --if-exists /tmp/banking.dump

```

12.2 Physical pgBackRest restore onto isolated toolbox storage

Use an isolated PVC/toolbox that mounts all three pgBackRest resources: pgbackrest-config, pgbackrest-repo-secret and pgbackrest-gcs-key. The restore command must continue to use config /etc/pgbackrest/pgbackrest.conf and stanza postgres-ha. To restore the latest eligible backup, omit --set. Use --set only with a real backup label. Do not use --set=latest.

```

kubectl -n "$NS" exec pgbackrest-restore-toolbox -c postgres -- sh -lc 'mkdir -p
  /restore/pgdata; export PGBACKREST_REPO1_CIPHER_PASS="$(cat
  /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
  --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --pg1-path=/restore/pgdata
  restore'

```

13. Lab 9 — Point-in-Time Recovery (PITR)

PITR also uses isolated storage and the same config, stanza, cipher Secret and GCS key.

```
TARGET_TIME="2026-09-29 12:00:00+05:30"
```

Set **TARGET_TIME** to a timestamp after the wanted transactions and before the unwanted ones.

```

kubectl -n "$NS" exec pgbackrest-pitr-toolbox -c postgres -- env TARGET_TIME="$TARGET_TIME" sh
  -lc 'mkdir -p /restore/pgdata; export PGBACKREST_REPO1_CIPHER_PASS="$(cat
  /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest
  --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --pg1-path=/restore/pgdata
  --type=time --target="$TARGET_TIME" --target-action=promote restore'

```

Expected: rows committed before **TARGET_TIME** are present and later unwanted changes are absent.

14. Patroni Failover Considerations During Backup

- Rediscover PRIMARY before stanza-create, check and every backup.
- The acid-postgres-ha Service follows the current primary; direct pod names do not.
- After failover, inspect pgBackRest info before deciding whether another backup is needed.
- Never edit active PGDATA manually.

15. Optional Disposable backup-lab (safe pgBackRest playground)

If you want to practice full / incr / diff / PITR without touching the live Patroni image or PVCs, create an independent postgres:18 + pgBackRest StatefulSet. This is the disposable lab from the Kubernetes backup-lab manual.

```
kubectl get storageclass
```

Create a password Secret first:

```

kubectl -n postgres-ha create secret generic backup-lab-auth
  --from-literal=password="ChangeMe-Lab-Only" --dry-run=client -o yaml | kubectl apply -f -

```

Apply backup-lab.yaml with a 3Gi repo PVC, ConfigMap stanza lab, and a single postgres:18-trixie StatefulSet that installs pgBackRest at start and sets archive_command to pgbackrest --stanza=lab archive-push %p. Practice stanza-create, full/incr/diff, restore and PITR only on that pod's data volume.

Do not mount live acid-postgres-ha PVCs into backup-lab.

16. Validation, Monitoring, Troubleshooting and Cleanup

16.1 Troubleshooting appendix — errors already seen

repo1-cipher-pass must have a value	Export PGBACKREST_REPO1_CIPHER_PASS inside the same kubectl exec command.
metadata.google.internal not found	Use GCS key-type=service with the mounted JSON key (Docker Kubernetes).
missing stanza path	Run stanza-create on the current primary.
unable to find primary cluster	You ran stanza-create on a replica; rediscover PRIMARY.
archive_command '/bin/true'	Apply section 7.10 and verify archive_command.
WAL segment was not archived before timeout	Verify archive_command uses /etc/pgbackrest/pgbackrest.conf + stanza postgres-ha, then run pgBackRest check before retrying backup.
old /home/postgres/pgdata/pgbackrest.conf or stanza acid-postgres-ha	Old archive identity is still active. Apply section 7.10.
cat/etc/... not found	Use cat /etc/... with a space.
invalid JSON / newline in string literal	Validate patch files first with python3 -m json.tool.
pgBackRest not found in pod	Build and roll the custom image from Lab 4A. Do not apt-get inside the live pod.
docker push denied	docker login and confirm DOCKERHUB_USER owns the repository.
GCS 403 / AccessDenied	Confirm SA has roles/storage.objectAdmin on gs://\$BUCKET and the JSON key Secret is mounted.

16.2 Cleanup (training only — do not expire production backups by accident)

```
kubectl delete postgresql acid-recovery-test -n "$NS" --wait=false
```

Leave the live cluster, GCS bucket and cipher Secret intact unless the instructor explicitly requests teardown.

17. RPO / RTO and Suggested Backup Schedule

- WAL archiving: continuous.
- Full backup: weekly lab baseline; production cadence depends on RPO/RTO and data volume.
- Differential: useful between full backups for a shorter restore chain.
- Incremental: smaller backup windows but a longer dependency chain.
- Restore/PITR drills: run regularly on isolated storage.

18. Final Coverage Checklist

- Primary discovered dynamically
- Current live image recorded
- Custom Spilo + pgBackRest image built
- Image pushed to Docker Hub / registry
- Cluster rolled onto custom image; pgbackrest version OK on all members
- GCS bucket created / verified

- GCS service account and JSON key configured
- Original cipher Secret preserved
- pgBackRest config mounted on all pods
- GCS key mounted on all pods
- Stanza created on current primary
- archive_command uses /etc/pgbackrest/pgbackrest.conf
- archive_command uses stanza postgres-ha
- pgBackRest check succeeds
- WAL archiving succeeds before full backup
- Full backup succeeds
- Incremental backup succeeds
- Differential backup succeeds
- Isolated restore / PITR use same config, stanza and credentials
- Active Patroni PVCs untouched

End of unified lab manual. Sources: READABLE_COPYPASTE_FINAL (latest command edition), CORRECTED_6/8 (image build), CORRECTED_11 (Docker Kubernetes + GCS dry-run), Kubernetes_PostgreSQL18_Backup_Recovery_Lab_Manual (recovery-test cluster and disposable backup-lab).